

OmarRao/analyzer

No description provided.

Branch main

Language Python

License None

2026-06-10 09:16 UTC

RISK SCORE

100

OUT OF 100

CRITICAL

245

Critical Findings
ERROR severity

169

Warning
WARNING severity

Threat Level

Score

100/100

COMPOSITE RISK SCORE

CRITICAL

Low (0-19) Medium (20-44)

CRITICAL WEIGHT

2450

245 errors x 10 pts

WARNING WEIGHT

507

169 warnings x 3 pts

CVE WEIGHT

0

0 CVEs x 8 pts

RUNTIME WEIGHT

0

0 behaviors x 15 pts

PRIORITY FIX QUEUE (TOP 5 CRITICAL)

#	RULE	FILE	CWE	TECHNIQUE
1	missing-user	Dockerfile:12	CWE-250	N/A
2	tainted-sql-string	api/accounts.py:30	CWE-704	N/A
3	tainted-sql-string	api/accounts.py:42	CWE-704	N/A
4	tainted-sql-string	api/accounts.py:71	CWE-704	N/A

#	RULE	FILE	CWE	TECHNIQUE
5	tainted-sql-string	api\accounts.py:81	CWE-704	N/A

Attack Surface Analysis

SQL Injection
EXPOSED
CWE-89 / T1190
Initial Access

Command Injection
EXPOSED
CWE-78 / T1059
Execution

Cross-Site Scripting
DETECTED
CWE-79 / T1059.007
Execution

SSRF
EXPOSED
CWE-918 / T1090
Defense Evasion

Path Traversal
DETECTED
CWE-22 / T1083
Discovery

Hardcoded Credentials
CLEAR
CWE-798 / T1552.001
Credential Access

Weak Cryptography
DETECTED
CWE-327 / T1600
Defense Evasion

Insecure Deserialization
CLEAR
CWE-502 / T1059
Execution

Analysis Charts

SEVERITY DISTRIBUTION

Severity	Count
Critical	49
Warning	33

ATT&CK TECHNIQUE RADAR

Technique	Score
T1059	12
T1090	10
T1083	8
T1190	10
T1059.007	10
T1600	10

FINDINGS BY FILE

File	Critical	Warning
transactions.py	49	0
auth.py	46	0
admin.py	44	0
files.py	41	0
accounts.py	39	0
users.py	39	0
reports.py	34	0
app.py	33	0

SEVERITY PER FILE (TOP 6)

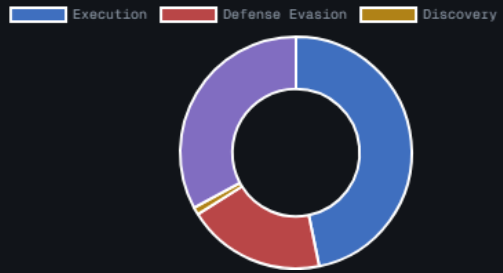
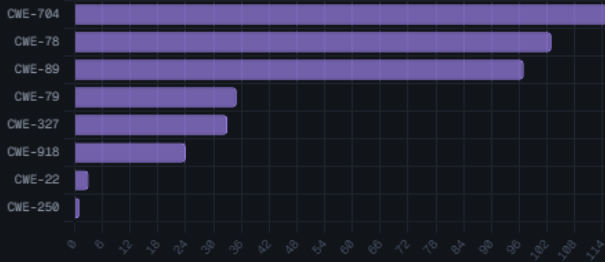
File	Critical	Warning
transactions.py	49	0
auth.py	46	0
admin.py	44	0
files.py	41	0
accounts.py	39	0
users.py	39	0

LANGUAGE RISK DISTRIBUTION

Language	Risk Score
Dockerfile	100
HTML	0
Python	0

CWE CATEGORY BREAKDOWN

TACTIC PHASE DISTRIBUTION



Repository Information



OmarRao/analyzer

No description provided.

Stars **1**

Forks **0**

Watchers **1**

Issues **0**

Size **48 KB**

DETAILS

Owner	OmarRao
Default Branch	main
License	None
Created	Jun 09, 2026
Last Push	Jun 09, 2026
Size	48 KB

LANGUAGE BREAKDOWN

Python	99.1%
HTML	0.8%
Dockerfile	0.1%

RECENT COMMITS

- [9e8357b](#) Add search service with dense SQLi and SSRF patterns
OmarRao / Jun 09, 2026
- [48a14a4](#) Add accounts API and notification service with additional vulnerabilities
OmarRao / Jun 09, 2026
- [0545297](#) Add 1000+ vulnerability VulnBank multi-module expansion
OmarRao / Jun 09, 2026
- [f42ba8c](#) Add VulnBank vulnerable app (CI workflow added separately)
OmarRao / Jun 09, 2026
- [253cb88](#) Add VulnBank: deliberately vulnerable Flask app for security scanner tests

CONTRIBUTORS

No contributor data.

FILE STRUCTURE

- Dockerfile
- README.md
- api
- app.py
- config.py
- jobs
- middleware
- models.py
- requirements.txt
- services
- templates
- utils.py
- utils

MITRE
ATT&CK
Coverage

6
techniques

T1059
OS Command Injection
Execution
103

T1090
SSRF
Defense Evasion
24

T1083
Path Traversal
Discovery
3

T1190
SQL Injection
Initial Access
97

T1059.007
XSS
Execution
35

T1600
Weak Cryptography
Defense Evasion

Vulnerability

Findings

414
total

All (414)

Critical (245)

Warning (169)

Filter by file, rule, CW

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
ERROR	<code>dockerfile.security.missing-user.missing-user</code> By not specifying a USER, a program in the container may run as 'root'. This is a security hazard. If an attac	<code>Dockerfile</code> L12	CWE-250
ERROR	<code>python.flask.security.injection.tainted-sql-string.tainted-sql-string</code> Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	<code>api\accounts.py</code> L30	CWE-704
ERROR	<code>python.flask.security.injection.tainted-sql-string.tainted-sql-string</code> Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	<code>api\accounts.py</code> L42	CWE-704
ERROR	<code>python.flask.security.injection.tainted-sql-string.tainted-sql-string</code> Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	<code>api\accounts.py</code> L71	CWE-704
ERROR	<code>python.flask.security.injection.tainted-sql-string.tainted-sql-string</code> Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	<code>api\accounts.py</code> L81	CWE-704
ERROR	<code>python.flask.security.injection.tainted-sql-string.tainted-sql-string</code> Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	<code>api\accounts.py</code> L94	CWE-704
ERROR	<code>python.flask.security.injection.tainted-sql-string.tainted-sql-string</code> Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	<code>api\accounts.py</code> L103	CWE-704
ERROR	<code>python.flask.security.injection.tainted-sql-string.tainted-sql-string</code> Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	<code>api\accounts.py</code> L112	CWE-704
ERROR	<code>python.flask.security.injection.subprocess-injection.subprocess-injection</code> Detected user input entering a 'subprocess' call unsafely. This could result in a command injection vulnerabil	<code>api\accounts.py</code> L123	CWE-78 T1059 Execution
ERROR	<code>python.lang.security.dangerous-subprocess-use.dangerous-subprocess-use</code>	<code>api\accounts.py</code> L124	CWE-78 T1059

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
	Detected subprocess function 'account_statement' with user controlled data. A malicious actor could leverage t		Execution
ERROR	python.lang.security.dangerous-subprocess-use.dangerous-subprocess-use Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this t	api\accounts.py L124	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	api\accounts.py L125	CWE-78 T1059 Execution
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\accounts.py L145	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\accounts.py L158	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\accounts.py L162	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\accounts.py L180	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\accounts.py L189	CWE-704
ERROR	python.django.security.injection.ssrf.ssrf-injection-urllib.ssrf-injection-urllib Data from request object is passed to a new server-side request. This could lead to a server-side request forg	api\accounts.py L197	CWE-918 T1090 Defense Evasion
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\accounts.py L203	CWE-704
ERROR	python.lang.security.dangerous-system-call.dangerous-system-call Found user-controlled data used in a system call. This could allow a malicious actor to execute commands. Use	api\accounts.py L213	CWE-78 T1059 Execution
ERROR	python.flask.security.injection.subprocess-injection.subprocess-injection Detected user input entering a 'subprocess' call unsafely. This could result in a command injection vulnerabil	api\accounts.py L223	CWE-78 T1059 Execution

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
ERROR	python.lang.security.dangerous-subprocess-use.dangerous-subprocess-use Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this t	api\accounts.py L224	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	api\accounts.py L225	CWE-78 T1059 Execution
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\accounts.py L236	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\accounts.py L248	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\admin.py L50	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\admin.py L60	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\admin.py L71	CWE-704
ERROR	python.flask.security.injection.subprocess-injection.subprocess-injection Detected user input entering a 'subprocess' call unsafely. This could result in a command injection vulnerabil	api\admin.py L79	CWE-78 T1059 Execution
ERROR	python.lang.security.dangerous-subprocess-use.dangerous-subprocess-use Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this t	api\admin.py L79	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	api\admin.py L79	CWE-78 T1059 Execution
ERROR	python.lang.security.dangerous-system-call.dangerous-system-call Found user-controlled data used in a system call. This could allow a malicious actor to execute commands. Use	api\admin.py L87	CWE-78 T1059 Execution
ERROR	python.flask.security.injection.subprocess-injection.subprocess-injection Detected user input entering a 'subprocess' call unsafely. This could result in a command injection vulnerabil	api\admin.py L97	CWE-78 T1059 Execution

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
ERROR	python.lang.security.dangerous-subprocess-use.dangerous-subprocess-use Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this t	api\admin.py L97	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	api\admin.py L97	CWE-78 T1059 Execution
ERROR	python.flask.security.injection.subprocess-injection.subprocess-injection Detected user input entering a 'subprocess' call unsafely. This could result in a command injection vulnerabil	api\admin.py L106	CWE-78 T1059 Execution
ERROR	python.lang.security.dangerous-subprocess-use.dangerous-subprocess-use Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this t	api\admin.py L107	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	api\admin.py L107	CWE-78 T1059 Execution
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\admin.py L125	CWE-704
ERROR	python.flask.security.injection.path-traversal-open.path-traversal-open Found request data in a call to 'open'. Ensure the request data is validated or sanitized, otherwise it could	api\admin.py L136	CWE-22 T1083 Discovery
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\admin.py L171	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\admin.py L174	CWE-704
ERROR	python.flask.security.injection.subprocess-injection.subprocess-injection Detected user input entering a 'subprocess' call unsafely. This could result in a command injection vulnerabil	api\admin.py L186	CWE-78 T1059 Execution
ERROR	python.lang.security.dangerous-subprocess-use.dangerous-subprocess-use Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this t	api\admin.py L187	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	api\admin.py L187	CWE-78 T1059 Execution
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string	api\admin.py L206	CWE-704

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
	Detected user input used to manually construct a SQL string. This is usually bad practice because manual const		
ERROR	python.lang.security.dangerous-system-call.dangerous-system-call Found user-controlled data used in a system call. This could allow a malicious actor to execute commands. Use	api\admin.py L215	CWE-78 T1059 Execution
ERROR	python.flask.security.injection.subprocess-injection.subprocess-injection Detected user input entering a 'subprocess' call unsafely. This could result in a command injection vulnerabil	api\admin.py L223	CWE-78 T1059 Execution
ERROR	python.lang.security.dangerous-subprocess-use.dangerous-subprocess-use Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this t	api\admin.py L224	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	api\admin.py L224	CWE-78 T1059 Execution
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\admin.py L254	CWE-704
ERROR	python.flask.security.injection.subprocess-injection.subprocess-injection Detected user input entering a 'subprocess' call unsafely. This could result in a command injection vulnerabil	api\admin.py L265	CWE-78 T1059 Execution
ERROR	python.lang.security.dangerous-subprocess-use.dangerous-subprocess-use Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this t	api\admin.py L266	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	api\admin.py L266	CWE-78 T1059 Execution
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\auth.py L36	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\auth.py L42	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\auth.py L63	CWE-704

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\auth.py L87	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\auth.py L90	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\auth.py L102	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\auth.py L116	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\auth.py L121	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\auth.py L135	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\auth.py L139	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\auth.py L149	CWE-704
ERROR	python.django.security.injection.ssrf.ssrf-injection-urllib.ssrf-injection-urllib Data from request object is passed to a new server-side request. This could lead to a server-side request forg	api\auth.py L157	CWE-918 T1090 Defense Evasion
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\auth.py L178	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\auth.py L191	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\auth.py L194	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string	api\auth.py L208	CWE-704

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
	Detected user input used to manually construct a SQL string. This is usually bad practice because manual const		
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\auth.py L218	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\auth.py L221	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\auth.py L233	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\files.py L59	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\files.py L69	CWE-704
ERROR	python.flask.security.injection.subprocess-injection.subprocess-injection Detected user input entering a 'subprocess' call unsafely. This could result in a command injection vulnerabil	api\files.py L74	CWE-78 T1059 Execution
ERROR	python.lang.security.dangerous-subprocess-use.dangerous-subprocess-use Detected subprocess function 'delete_file' with user controlled data. A malicious actor could leverage this to	api\files.py L74	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	api\files.py L74	CWE-78 T1059 Execution
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\files.py L75	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\files.py L86	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\files.py L101	CWE-704

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
ERROR	python.flask.security.injection.subprocess-injection.subprocess-injection Detected user input entering a 'subprocess' call unsafely. This could result in a command injection vulnerabil	api\files.py L123	CWE-78 T1059 Execution
ERROR	python.lang.security.dangerous-subprocess-use.dangerous-subprocess-use Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this t	api\files.py L124	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	api\files.py L124	CWE-78 T1059 Execution
ERROR	python.flask.security.injection.subprocess-injection.subprocess-injection Detected user input entering a 'subprocess' call unsafely. This could result in a command injection vulnerabil	api\files.py L134	CWE-78 T1059 Execution
ERROR	python.lang.security.dangerous-subprocess-use.dangerous-subprocess-use Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this t	api\files.py L135	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	api\files.py L135	CWE-78 T1059 Execution
ERROR	python.lang.security.dangerous-system-call.dangerous-system-call Found user-controlled data used in a system call. This could allow a malicious actor to execute commands. Use	api\files.py L146	CWE-78 T1059 Execution
ERROR	python.flask.security.injection.subprocess-injection.subprocess-injection Detected user input entering a 'subprocess' call unsafely. This could result in a command injection vulnerabil	api\files.py L156	CWE-78 T1059 Execution
ERROR	python.lang.security.dangerous-subprocess-use.dangerous-subprocess-use Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this t	api\files.py L157	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	api\files.py L157	CWE-78 T1059 Execution
ERROR	python.flask.security.injection.subprocess-injection.subprocess-injection Detected user input entering a 'subprocess' call unsafely. This could result in a command injection vulnerabil	api\files.py L166	CWE-78 T1059 Execution
ERROR	python.lang.security.dangerous-subprocess-use.dangerous-subprocess-use Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this t	api\files.py L167	CWE-78 T1059 Execution

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	api\files.py L167	CWE-78 T1059 Execution
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\files.py L178	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\files.py L188	CWE-704
ERROR	python.flask.security.injection.subprocess-injection.subprocess-injection Detected user input entering a 'subprocess' call unsafely. This could result in a command injection vulnerabil	api\files.py L191	CWE-78 T1059 Execution
ERROR	python.lang.security.dangerous-subprocess-use.dangerous-subprocess-use Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this t	api\files.py L191	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	api\files.py L191	CWE-78 T1059 Execution
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\files.py L192	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\files.py L202	CWE-704
ERROR	python.flask.security.injection.subprocess-injection.subprocess-injection Detected user input entering a 'subprocess' call unsafely. This could result in a command injection vulnerabil	api\files.py L206	CWE-78 T1059 Execution
ERROR	python.lang.security.dangerous-subprocess-use.dangerous-subprocess-use Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this t	api\files.py L207	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	api\files.py L207	CWE-78 T1059 Execution
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\loans.py L28	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\loans.py L38	CWE-704

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
	Detected user input used to manually construct a SQL string. This is usually bad practice because manual const		
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\loans.py L68	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\loans.py L79	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\loans.py L90	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\loans.py L92	CWE-704
ERROR	python.django.security.injection.ssrfs.ssrfinjection-urllib.ssrfinjection-urllib Data from request object is passed to a new server-side request. This could lead to a server-side request forg	api\loans.py L100	CWE-918 T1090 Defense Evasion
ERROR	python.django.security.injection.ssrfs.ssrfinjection-urllib.ssrfinjection-urllib Data from request object is passed to a new server-side request. This could lead to a server-side request forg	api\loans.py L101	CWE-918 T1090 Defense Evasion
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\loans.py L108	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\loans.py L119	CWE-704
ERROR	python.flask.security.injection.subprocessinjection.subprocessinjection Detected user input entering a `subprocess` call unsafely. This could result in a command injection vulnerabil	api\loans.py L130	CWE-78 T1059 Execution
ERROR	python.lang.security.dangerous-subprocess-use.dangerous-subprocess-use Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this t	api\loans.py L131	CWE-78 T1059 Execution
ERROR	python.lang.security.dangerous-subprocess-use.dangerous-subprocess-use Detected subprocess function 'loan_statement' with user controlled data. A malicious actor could leverage this	api\loans.py L131	CWE-78 T1059 Execution

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	api\loans.py L132	CWE-78 T1059 Execution
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\loans.py L144	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\loans.py L157	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\payments.py L33	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\payments.py L45	CWE-704
ERROR	python.django.security.injection.ssrf.ssrf-injection-urllib.ssrf-injection-urllib Data from request object is passed to a new server-side request. This could lead to a server-side request forg	api\payments.py L52	CWE-918 T1090 Defense Evasion
ERROR	python.django.security.injection.ssrf.ssrf-injection-urllib.ssrf-injection-urllib Data from request object is passed to a new server-side request. This could lead to a server-side request forg	api\payments.py L53	CWE-918 T1090 Defense Evasion
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\payments.py L77	CWE-704
ERROR	python.django.security.injection.ssrf.ssrf-injection-urllib.ssrf-injection-urllib Data from request object is passed to a new server-side request. This could lead to a server-side request forg	api\payments.py L86	CWE-918 T1090 Defense Evasion
ERROR	python.django.security.injection.ssrf.ssrf-injection-urllib.ssrf-injection-urllib Data from request object is passed to a new server-side request. This could lead to a server-side request forg	api\payments.py L87	CWE-918 T1090 Defense Evasion
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\payments.py L106	CWE-704
ERROR	python.django.security.injection.ssrf.ssrf-injection-urllib.ssrf-injection-urllib Data from request object is passed to a new server-side request. This could lead to a server-side request forg	api\payments.py L114	CWE-918 T1090 Defense Evasion

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
ERROR	python.django.security.injection.ssrf.ssrf-injection-urllib.ssrf-injection-urllib Data from request object is passed to a new server-side request. This could lead to a server-side request forg	api\payments.py L115	CWE-918 T1090 Defense Evasion
ERROR	python.django.security.injection.ssrf.ssrf-injection-urllib.ssrf-injection-urllib Data from request object is passed to a new server-side request. This could lead to a server-side request forg	api\payments.py L116	CWE-918 T1090 Defense Evasion
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\payments.py L126	CWE-704
ERROR	python.flask.security.injection.subprocess-injection.subprocess-injection Detected user input entering a 'subprocess' call unsafely. This could result in a command injection vulnerabil	api\payments.py L151	CWE-78 T1059 Execution
ERROR	python.lang.security.dangerous-subprocess-use.dangerous-subprocess-use Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this t	api\payments.py L152	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	api\payments.py L152	CWE-78 T1059 Execution
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\payments.py L177	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\payments.py L205	CWE-704
ERROR	python.lang.security.dangerous-system-call.dangerous-system-call Found user-controlled data used in a system call. This could allow a malicious actor to execute commands. Use	api\payments.py L221	CWE-78 T1059 Execution
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\reports.py L31	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\reports.py L42	CWE-704
ERROR	python.flask.security.injection.subprocess-injection.subprocess-injection Detected user input entering a 'subprocess' call unsafely. This could result in a command injection vulnerabil	api\reports.py L55	CWE-78 T1059 Execution

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
ERROR	python.lang.security.dangerous-subprocess-use.dangerous-subprocess-use Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this t	api\reports.py L56	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	api\reports.py L58	CWE-78 T1059 Execution
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\reports.py L83	CWE-704
ERROR	python.lang.security.dangerous-system-call.dangerous-system-call Found user-controlled data used in a system call. This could allow a malicious actor to execute commands. Use	api\reports.py L112	CWE-78 T1059 Execution
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\reports.py L141	CWE-704
ERROR	python.django.security.injection.ssrf.ssrf-injection-urllib.ssrf-injection-urllib Data from request object is passed to a new server-side request. This could lead to a server-side request forg	api\reports.py L148	CWE-918 T1090 Defense Evasion
ERROR	python.django.security.injection.ssrf.ssrf-injection-urllib.ssrf-injection-urllib Data from request object is passed to a new server-side request. This could lead to a server-side request forg	api\reports.py L149	CWE-918 T1090 Defense Evasion
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\reports.py L162	CWE-704
ERROR	python.django.security.injection.ssrf.ssrf-injection-urllib.ssrf-injection-urllib Data from request object is passed to a new server-side request. This could lead to a server-side request forg	api\reports.py L171	CWE-918 T1090 Defense Evasion
ERROR	python.django.security.injection.ssrf.ssrf-injection-urllib.ssrf-injection-urllib Data from request object is passed to a new server-side request. This could lead to a server-side request forg	api\reports.py L172	CWE-918 T1090 Defense Evasion
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\reports.py L182	CWE-704
ERROR	python.flask.security.injection.subprocess-injection.subprocess-injection Detected user input entering a 'subprocess' call unsafely. This could result in a command injection vulnerabil	api\reports.py L190	CWE-78 T1059 Execution

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
ERROR	python.lang.security.dangerous-subprocess-use.dangerous-subprocess-use Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this t	api\reports.py L191	CWE-78 T1059 Execution
ERROR	python.lang.security.dangerous-subprocess-use.dangerous-subprocess-use Detected subprocess function 'render_template' with user controlled data. A malicious actor could leverage thi	api\reports.py L191	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	api\reports.py L192	CWE-78 T1059 Execution
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\transactions.py L30	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\transactions.py L45	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\transactions.py L60	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\transactions.py L61	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\transactions.py L64	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\transactions.py L65	CWE-704
ERROR	python.django.security.injection.ssrf.ssrf-injection-urllib.ssrf-injection-urllib Data from request object is passed to a new server-side request. This could lead to a server-side request forg	api\transactions.py L76	CWE-918 T1090 Defense Evasion
ERROR	python.django.security.injection.ssrf.ssrf-injection-urllib.ssrf-injection-urllib Data from request object is passed to a new server-side request. This could lead to a server-side request forg	api\transactions.py L77	CWE-918 T1090 Defense Evasion
ERROR	python.django.security.injection.ssrf.ssrf-injection-urllib.ssrf-injection-urllib Data from request object is passed to a new server-side request. This could lead to a server-side request forg	api\transactions.py L78	CWE-918 T1090 Defense Evasion

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
ERROR	python.django.security.injection.ssrf.ssrf-injection-urllib.ssrf-injection-urllib Data from request object is passed to a new server-side request. This could lead to a server-side request forg	api\transactions.py L79	CWE-918 T1090 Defense Evasion
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\transactions.py L96	CWE-704
ERROR	python.flask.security.injection.subprocess-injection.subprocess-injection Detected user input entering a 'subprocess' call unsafely. This could result in a command injection vulnerabil	api\transactions.py L113	CWE-78 T1059 Execution
ERROR	python.lang.security.dangerous-subprocess-use.dangerous-subprocess-use Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this t	api\transactions.py L114	CWE-78 T1059 Execution
ERROR	python.lang.security.dangerous-subprocess-use.dangerous-subprocess-use Detected subprocess function 'get_receipt' with user controlled data. A malicious actor could leverage this to	api\transactions.py L114	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	api\transactions.py L114	CWE-78 T1059 Execution
ERROR	python.lang.security.dangerous-system-call.dangerous-system-call Found user-controlled data used in a system call. This could allow a malicious actor to execute commands. Use	api\transactions.py L133	CWE-78 T1059 Execution
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\transactions.py L155	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\transactions.py L156	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\transactions.py L158	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\transactions.py L168	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\transactions.py L180	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\transactions.py L202	CWE-704

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
	Detected user input used to manually construct a SQL string. This is usually bad practice because manual const		
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\transactions.py L213	CWE-704
ERROR	python.flask.security.injection.subprocess-injection.subprocess-injection Detected user input entering a 'subprocess' call unsafely. This could result in a command injection vulnerabil	api\transactions.py L224	CWE-78 T1059 Execution
ERROR	python.lang.security.dangerous-subprocess-use.dangerous-subprocess-use Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this t	api\transactions.py L225	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	api\transactions.py L225	CWE-78 T1059 Execution
ERROR	python.django.security.injection.ssrfs.ssrfinjection-urllib.ssrfinjection-urllib Data from request object is passed to a new server-side request. This could lead to a server-side request forg	api\transactions.py L232	CWE-918 T1090 Defense Evasion
ERROR	python.django.security.injection.ssrfs.ssrfinjection-urllib.ssrfinjection-urllib Data from request object is passed to a new server-side request. This could lead to a server-side request forg	api\transactions.py L233	CWE-918 T1090 Defense Evasion
ERROR	python.django.security.injection.ssrfs.ssrfinjection-urllib.ssrfinjection-urllib Data from request object is passed to a new server-side request. This could lead to a server-side request forg	api\transactions.py L234	CWE-918 T1090 Defense Evasion
ERROR	python.django.security.injection.ssrfs.ssrfinjection-urllib.ssrfinjection-urllib Data from request object is passed to a new server-side request. This could lead to a server-side request forg	api\transactions.py L235	CWE-918 T1090 Defense Evasion
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\transactions.py L248	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\transactions.py L262	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\transactions.py L266	CWE-704

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\users.py L30	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\users.py L42	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\users.py L56	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\users.py L60	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\users.py L74	CWE-704
ERROR	python.flask.security.injection.subprocess-injection.subprocess-injection Detected user input entering a 'subprocess' call unsafely. This could result in a command injection vulnerabil	api\users.py L90	CWE-78 T1059 Execution
ERROR	python.lang.security.dangerous-subprocess-use.dangerous-subprocess-use Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this t	api\users.py L91	CWE-78 T1059 Execution
ERROR	python.lang.security.dangerous-subprocess-use.dangerous-subprocess-use Detected subprocess function 'export_user_data' with user controlled data. A malicious actor could leverage th	api\users.py L91	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	api\users.py L91	CWE-78 T1059 Execution
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\users.py L126	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\users.py L135	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\users.py L145	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\users.py L155	CWE-704

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
	Detected user input used to manually construct a SQL string. This is usually bad practice because manual const		
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\users.py L158	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\users.py L167	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\users.py L170	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\users.py L188	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\users.py L210	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	api\users.py L219	CWE-704
ERROR	python.lang.security.dangerous-system-call.dangerous-system-call Found user-controlled data used in a system call. This could allow a malicious actor to execute commands. Use	api\users.py L229	CWE-78 T1059 Execution
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	app.py L72	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	app.py L111	CWE-704
ERROR	python.flask.security.injection.subprocess-injection.subprocess-injection Detected user input entering a 'subprocess' call unsafely. This could result in a command injection vulnerabil	app.py L125	CWE-78 T1059 Execution
ERROR	python.lang.security.dangerous-subprocess-use.dangerous-subprocess-use Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this t	app.py L125	CWE-78 T1059 Execution

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	app.py L125	CWE-78 T1059 Execution
ERROR	python.lang.security.dangerous-system-call.dangerous-system-call Found user-controlled data used in a system call. This could allow a malicious actor to execute commands. Use	app.py L145	CWE-78 T1059 Execution
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	app.py L155	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	app.py L175	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	app.py L177	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	app.py L178	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	app.py L179	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	app.py L206	CWE-704
ERROR	python.django.security.injection.ssrf.ssrf-injection-urllib.ssrf-injection-urllib Data from request object is passed to a new server-side request. This could lead to a server-side request forg	app.py L218	CWE-918 T1090 Defense Evasion
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	jobs\scheduled.py L26	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	jobs\scheduled.py L48	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	jobs\scheduled.py L69	CWE-78 T1059 Execution

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	jobs\scheduled.py L111	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	jobs\scheduled.py L139	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	jobs\scheduled.py L150	CWE-78 T1059 Execution
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	middleware\auth.py L31	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	middleware\auth.py L52	CWE-704
ERROR	python.flask.security.injection.tainted-sql-string.tainted-sql-string Detected user input used to manually construct a SQL string. This is usually bad practice because manual const	middleware\auth.py L68	CWE-704
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	services\crypto_service.py L158	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	services\crypto_service.py L167	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	services\email.py L47	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	services\email.py L140	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	services\email.py L152	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	services\email.py L169	CWE-78 T1059 Execution

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	services\logger.py L67	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	services\logger.py L144	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	services\logger.py L152	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	services\notification.py L132	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	services\search.py L150	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	utils\formatters.py L86	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	utils\formatters.py L144	CWE-78 T1059 Execution
ERROR	python.lang.security.audit.subprocess-shell-true.subprocess-shell-true Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn t	utils\validators.py L157	CWE-78 T1059 Execution
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\accounts.py L36	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\accounts.py L91	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\accounts.py L110	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute	api\accounts.py L139	CWE-89 T1190 Initial Access

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore		
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\accounts.py L140	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\accounts.py L141	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\accounts.py L154	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\accounts.py L155	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\accounts.py L186	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\accounts.py L196	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\accounts.py L244	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\accounts.py L245	CWE-89 T1190 Initial Access
WARNING	python.flask.security.audit.directly-returned-format-string.directly-returned-format-string Detected Flask route directly returning a formatted string. This is subject to cross-site scripting if user in	api\accounts.py L252	CWE-79 T1059.007 Execution
WARNING	python.django.security.injection.raw-html-format.raw-html-format Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure	api\accounts.py L252	CWE-79 T1059.007 Execution

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
WARNING	python.flask.security.injection.raw-html-concat.raw-html-format Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure	api\accounts.py L252	CWE-79 T1059.007 Execution
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\admin.py L48	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\admin.py L58	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\admin.py L67	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\admin.py L68	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\admin.py L123	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\admin.py L124	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\admin.py L165	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\admin.py L166	CWE-89 T1190 Initial Access
WARNING	python.flask.security.audit.directly-returned-format-string.directly-returned-format-string Detected Flask route directly returning a formatted string. This is subject to cross-site scripting if user in	api\admin.py L178	CWE-79 T1059.007 Execution
WARNING	python.django.security.injection.raw-html-format.raw-html-format Detected user input flowing into a manually constructed HTML string. You may be accidentally	api\admin.py L178	CWE-79 T1059.007 Execution

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
	bypassing secure		
WARNING	python.flask.security.injection.raw-html-concat.raw-html-format Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure	api\admin.py L178	CWE-79 T1059.007 Execution
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\admin.py L195	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\admin.py L249	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\admin.py L250	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\admin.py L251	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\auth.py L30	CWE-89 T1190 Initial Access
WARNING	python.lang.security.insecure-hash-algorithms-md5.insecure-hash-algorithm-md5 Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not	api\auth.py L36	CWE-327 T1600 Defense Evasion
WARNING	python.lang.security.audit.md5-used-as-password.md5-used-as-password It looks like MD5 is used as a password hash. MD5 is not considered a secure password hash because it can be c	api\auth.py L36	CWE-327 T1600 Defense Evasion
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\auth.py L55	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\auth.py L56	CWE-89 T1190 Initial Access

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
WARNING	python.lang.security.insecure-hash-algorithms-md5.insecure-hash-algorithm-md5 Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not	api\auth.py L68	CWE-327 T1600 Defense Evasion
WARNING	python.lang.security.audit.md5-used-as-password.md5-used-as-password It looks like MD5 is used as a password hash. MD5 is not considered a secure password hash because it can be c	api\auth.py L68	CWE-327 T1600 Defense Evasion
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\auth.py L82	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\auth.py L83	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\auth.py L83	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\auth.py L98	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\auth.py L110	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\auth.py L111	CWE-89 T1190 Initial Access
WARNING	python.lang.security.insecure-hash-algorithms-md5.insecure-hash-algorithm-md5 Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not	api\auth.py L120	CWE-327 T1600 Defense Evasion
WARNING	python.lang.security.audit.md5-used-as-password.md5-used-as-password It looks like MD5 is used as a password hash. MD5 is not considered a secure password hash because it can be c	api\auth.py L120	CWE-327 T1600 Defense Evasion
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\auth.py L129	CWE-89 T1190 Initial Access

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\auth.py L129	CWE-89 T1190 Initial Access
WARNING	python.lang.security.insecure-hash-algorithms-md5.insecure-hash-algorithm-md5 Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not	api\auth.py L133	CWE-327 T1600 Defense Evasion
WARNING	python.lang.security.audit.md5-used-as-password.md5-used-as-password It looks like MD5 is used as a password hash. MD5 is not considered a secure password hash because it can be c	api\auth.py L133	CWE-327 T1600 Defense Evasion
WARNING	python.lang.security.insecure-hash-algorithms-md5.insecure-hash-algorithm-md5 Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not	api\auth.py L138	CWE-327 T1600 Defense Evasion
WARNING	python.lang.security.audit.md5-used-as-password.md5-used-as-password It looks like MD5 is used as a password hash. MD5 is not considered a secure password hash because it can be c	api\auth.py L138	CWE-327 T1600 Defense Evasion
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\auth.py L147	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\auth.py L188	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\auth.py L188	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\auth.py L202	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\auth.py L216	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\auth.py L229	CWE-89 T1190 Initial Access

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\files.py L82	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\files.py L96	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\files.py L97	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.path-traversal.path-traversal-open.path-traversal-open Found request data in a call to 'open'. Ensure the request data is validated or sanitized, otherwise it could	api\files.py L109	CWE-22 T1083 Discovery
WARNING	python.flask.security.audit.directly-returned-format-string.directly-returned-format-string Detected Flask route directly returning a formatted string. This is subject to cross-site scripting if user in	api\files.py L115	CWE-79 T1059.007 Execution
WARNING	python.django.security.injection.raw-html-format.raw-html-format Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure	api\files.py L115	CWE-79 T1059.007 Execution
WARNING	python.flask.security.injection.raw-html-concat.raw-html-format Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure	api\files.py L115	CWE-79 T1059.007 Execution
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\files.py L174	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\files.py L175	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\files.py L199	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and	api\loans.py L24	CWE-89 T1190 Initial Access

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
	therefore		
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\loans.py L77	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\loans.py L86	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\loans.py L86	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\loans.py L87	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\loans.py L100	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\loans.py L115	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\loans.py L153	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\loans.py L154	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\payments.py L28	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\payments.py L100	CWE-89 T1190 Initial Access

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\reports.py L26	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\reports.py L120	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\reports.py L121	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.raw-html-format.raw-html-format Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure	api\reports.py L127	CWE-79 T1059.007 Execution
WARNING	python.flask.security.injection.raw-html-concat.raw-html-format Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure	api\reports.py L127	CWE-79 T1059.007 Execution
WARNING	python.django.security.injection.raw-html-format.raw-html-format Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure	api\reports.py L129	CWE-79 T1059.007 Execution
WARNING	python.flask.security.injection.raw-html-concat.raw-html-format Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure	api\reports.py L129	CWE-79 T1059.007 Execution
WARNING	python.flask.security.audit.directly-returned-format-string.directly-returned-format-string Detected Flask route directly returning a formatted string. This is subject to cross-site scripting if user in	api\reports.py L131	CWE-79 T1059.007 Execution
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\reports.py L136	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\reports.py L137	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\reports.py L138	CWE-89 T1190 Initial Access

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\reports.py L158	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\reports.py L159	CWE-89 T1190 Initial Access
WARNING	python.flask.security.audit.directly-returned-format-string.directly-returned-format-string Detected Flask route directly returning a formatted string. This is subject to cross-site scripting if user in	api\reports.py L166	CWE-79 T1059.007 Execution
WARNING	python.django.security.injection.raw-html-format.raw-html-format Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure	api\reports.py L166	CWE-79 T1059.007 Execution
WARNING	python.flask.security.injection.raw-html-concat.raw-html-format Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure	api\reports.py L166	CWE-79 T1059.007 Execution
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\transactions.py L53	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\transactions.py L53	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\transactions.py L54	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\transactions.py L54	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\transactions.py L55	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\transactions.py L55	CWE-89 T1190 Initial Access

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
	therefore		
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\transactions.py L166	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\transactions.py L175	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\transactions.py L176	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\transactions.py L177	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\transactions.py L201	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\transactions.py L244	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\transactions.py L245	CWE-89 T1190 Initial Access
WARNING	python.flask.security.audit.directly-returned-format-string.directly-returned-format-string Detected Flask route directly returning a formatted string. This is subject to cross-site scripting if user in	api\transactions.py L252	CWE-79 T1059.007 Execution
WARNING	python.django.security.injection.raw-html-format.raw-html-format Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure	api\transactions.py L252	CWE-79 T1059.007 Execution
WARNING	python.flask.security.injection.raw-html-concat.raw-html-format Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure	api\transactions.py L252	CWE-79 T1059.007 Execution
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute	api\users.py L38	CWE-89 T1190

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore		Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\users.py L39	CWE-89 T1190 Initial Access
WARNING	python.flask.security.audit.directly-returned-format-string.directly-returned-format-string Detected Flask route directly returning a formatted string. This is subject to cross-site scripting if user in	api\users.py L44	CWE-79 T1059.007 Execution
WARNING	python.django.security.injection.raw-html-format.raw-html-format Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure	api\users.py L44	CWE-79 T1059.007 Execution
WARNING	python.flask.security.injection.raw-html-concat.raw-html-format Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure	api\users.py L44	CWE-79 T1059.007 Execution
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\users.py L51	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\users.py L52	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\users.py L53	CWE-89 T1190 Initial Access
WARNING	python.lang.security.insecure-hash-algorithms-md5.insecure-hash-algorithm-md5 Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not	api\users.py L111	CWE-327 T1600 Defense Evasion
WARNING	python.lang.security.audit.md5-used-as-password.md5-used-as-password It looks like MD5 is used as a password hash. MD5 is not considered a secure password hash because it can be c	api\users.py L111	CWE-327 T1600 Defense Evasion
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\users.py L122	CWE-89 T1190 Initial Access

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\users.py L123	CWE-89 T1190 Initial Access
WARNING	python.flask.security.audit.directly-returned-format-string.directly-returned-format-string Detected Flask route directly returning a formatted string. This is subject to cross-site scripting if user in	api\users.py L129	CWE-79 T1059.007 Execution
WARNING	python.django.security.injection.raw-html-format.raw-html-format Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure	api\users.py L129	CWE-79 T1059.007 Execution
WARNING	python.flask.security.injection.raw-html-concat.raw-html-format Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure	api\users.py L129	CWE-79 T1059.007 Execution
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\users.py L143	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\users.py L153	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\users.py L166	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	api\users.py L216	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	app.py L67	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	app.py L68	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	app.py L105	CWE-89 T1190 Initial Access

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
	therefore		
WARNING	python.flask.security.audit.directly-returned-format-string.directly-returned-format-string Detected Flask route directly returning a formatted string. This is subject to cross-site scripting if user in	app.py L116	CWE-79 T1059.007 Execution
WARNING	python.django.security.injection.raw-html-format.raw-html-format Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure	app.py L116	CWE-79 T1059.007 Execution
WARNING	python.flask.security.injection.raw-html-concat.raw-html-format Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure	app.py L116	CWE-79 T1059.007 Execution
WARNING	python.flask.security.audit.directly-returned-format-string.directly-returned-format-string Detected Flask route directly returning a formatted string. This is subject to cross-site scripting if user in	app.py L126	CWE-79 T1059.007 Execution
WARNING	python.django.security.injection.raw-html-format.raw-html-format Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure	app.py L126	CWE-79 T1059.007 Execution
WARNING	python.flask.security.injection.raw-html-concat.raw-html-format Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure	app.py L126	CWE-79 T1059.007 Execution
WARNING	python.django.security.injection.path-traversal.path-traversal-open.path-traversal-open Found request data in a call to 'open'. Ensure the request data is validated or sanitized, otherwise it could	app.py L132	CWE-22 T1083 Discovery
WARNING	python.flask.security.audit.directly-returned-format-string.directly-returned-format-string Detected Flask route directly returning a formatted string. This is subject to cross-site scripting if user in	app.py L136	CWE-79 T1059.007 Execution
WARNING	python.django.security.injection.raw-html-format.raw-html-format Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure	app.py L136	CWE-79 T1059.007 Execution
WARNING	python.flask.security.injection.raw-html-concat.raw-html-format Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure	app.py L136	CWE-79 T1059.007 Execution
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	app.py L170	CWE-89 T1190 Initial Access

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	app.py L170	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	app.py L171	CWE-89 T1190 Initial Access
WARNING	python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore	app.py L199	CWE-89 T1190 Initial Access
WARNING	python.lang.security.insecure-hash-algorithms-md5.insecure-hash-algorithm-md5 Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not	app.py L203	CWE-327 T1600 Defense Evasion
WARNING	python.flask.security.audit.app-run-param-config.avoid_app_run_with_bad_host Running flask app with host 0.0.0.0 could expose the server publicly.	app.py L226	CWE-668
WARNING	python.flask.security.audit.debug-enabled.debug-enabled Detected Flask app with debug=True. Do not deploy to production with this flag enabled as it will leak sensiti	app.py L226	CWE-489
WARNING	python.lang.security.insecure-hash-algorithms-md5.insecure-hash-algorithm-md5 Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not	middleware\auth.py L139	CWE-327 T1600 Defense Evasion
WARNING	python.lang.security.insecure-hash-algorithms-md5.insecure-hash-algorithm-md5 Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not	models.py L59	CWE-327 T1600 Defense Evasion
WARNING	python.lang.security.audit.md5-used-as-password.md5-used-as-password It looks like MD5 is used as a password hash. MD5 is not considered a secure password hash because it can be c	models.py L59	CWE-327 T1600 Defense Evasion
WARNING	python.lang.security.insecure-hash-algorithms-md5.insecure-hash-algorithm-md5 Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not	models.py L79	CWE-327 T1600 Defense Evasion
WARNING	python.lang.security.audit.md5-used-as-password.md5-used-as-password It looks like MD5 is used as a password hash. MD5 is not considered a secure password hash because it can be c	models.py L79	CWE-327 T1600 Defense Evasion
WARNING	python.lang.security.insecure-hash-algorithms-md5.insecure-hash-algorithm-md5 Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not	services\crypto_service.py L26	CWE-327 T1600 Defense Evasion

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
WARNING	python.lang.security.audit.md5-used-as-password.md5-used-as-password It looks like MD5 is used as a password hash. MD5 is not considered a secure password hash because it can be c	services\crypto_service.py L26	CWE-327 T1600 Defense Evasion
WARNING	python.lang.security.insecure-hash-algorithms.insecure-hash-algorithm-sha1 Detected SHA1 hash algorithm which is considered insecure. SHA1 is not collision resistant and is therefore no	services\crypto_service.py L31	CWE-327 T1600 Defense Evasion
WARNING	python.lang.security.insecure-hash-algorithms-md5.insecure-hash-algorithm-md5 Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not	services\crypto_service.py L36	CWE-327 T1600 Defense Evasion
WARNING	python.lang.security.insecure-hash-algorithms-md5.insecure-hash-algorithm-md5 Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not	services\crypto_service.py L51	CWE-327 T1600 Defense Evasion
WARNING	python.lang.security.insecure-hash-algorithms-md5.insecure-hash-algorithm-md5 Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not	services\crypto_service.py L77	CWE-327 T1600 Defense Evasion
WARNING	python.lang.security.insecure-hash-algorithms-md5.insecure-hash-algorithm-md5 Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not	services\crypto_service.py L82	CWE-327 T1600 Defense Evasion
WARNING	python.lang.security.insecure-hash-algorithms-md5.insecure-hash-algorithm-md5 Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not	services\crypto_service.py L88	CWE-327 T1600 Defense Evasion
WARNING	python.lang.security.insecure-hash-algorithms-md5.insecure-hash-algorithm-md5 Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not	services\crypto_service.py L96	CWE-327 T1600 Defense Evasion
WARNING	python.lang.security.insecure-hash-algorithms-md5.insecure-hash-algorithm-md5 Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not	services\crypto_service.py L141	CWE-327 T1600 Defense Evasion
WARNING	python.lang.security.insecure-hash-algorithms-md5.insecure-hash-algorithm-md5 Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not	services\crypto_service.py L151	CWE-327 T1600 Defense Evasion
WARNING	python.lang.security.audit.md5-used-as-password.md5-used-as-password It looks like MD5 is used as a password hash. MD5 is not considered a secure password hash because it can be c	services\crypto_service.py L151	CWE-327 T1600 Defense Evasion
WARNING	python.lang.security.insecure-hash-algorithms-md5.insecure-hash-algorithm-md5 Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not	utils.py L13	CWE-327 T1600 Defense Evasion

SEVERITY	RULE / DESCRIPTION	LOCATION	MITRE
WARNING	<code>python.lang.security.audit.md5-used-as-password.md5-used-as-password</code> It looks like MD5 is used as a password hash. MD5 is not considered a secure password hash because it can be c	<code>utils.py</code> L13	CWE-327 T1600 Defense Evasion
WARNING	<code>python.lang.security.insecure-hash-algorithms-md5.insecure-hash-algorithm-md5</code> Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not	<code>utils.py</code> L43	CWE-327 T1600 Defense Evasion

Dependency
CVEs

0

No known
CVEs
detected
in
dependencies.

Sandbox Runtime
Analysis

Sandbox not
enabled for
this scan.
Re-run with
Docker
Sandbox
option
active.